

utiliser impérativement les feuilles quadrillées fournies pour les schémas simplifiés

Une réponse non justifiée sera considérée comme fausse

On considère le processeur pipeline P9. Ce processeur a le même jeu d'instructions que le Mips. Il est construit autour d'un pipeline à 9 étages :

IF1, IF2, DEC1, DEC2, EXE1, EXE2, MEM1, MEM2, WBK

Le découpage en étages de ce pipeline a été obtenu à partir du pipeline du processeur Mips. Chacun des étages IF1, DEC1, EXE1 et MEM1 a été divisé en deux étages.

Cependant, le calcul de l'adresse de l'instruction suivante s'achève dans l'étage EXE2.

La réalisation contient tous les bypasses nécessaires pour prendre en charge les dépendances de données. Cependant il n'y a aucun bypass vers l'étage MEM1.

On souhaite étudier la performance d'exécution d'une fonction de calcul de distance sur cette réalisation.

On considère la fonction Dist. Cette fonction prend en entrée un tableau d'entiers x et calcule la valeur des éléments d'un tableau y tel que $y(i) = |x(i) - x(i+1)|$

```
void Dist (int *x, int *y, int size)
{
    int i;

    for (i=0 ; i!=(size-1) ; i++)
    {
        if (x[i] < x[i+1])
            y [i] = x [i+1] - x [i ];
        else
            y [i] = x [i ] - x [i+1];
    }
}
```

On a obtenu deux versions différentes pour le code de la boucle principale de Dist en assembleur Mips non pipeline.

On suppose qu'à l'entrée de la boucle le registre R4 contient l'adresse du tableau x, R5 l'adresse du tableau y, et R6 l'adresse du dernier élément du tableau x.

Version 1 :

```
Loop:   Lw      r8 , 0 (r4 )           ; x[i ]
        Lw      r9 , 4 (r4 )          ; x[i+1]
        Sub     r10, r8, r9
        Bgez    r10, _Endif
        Sub     r10, r9 , r8

_Endif: Sw      r10, 0 (r5 )           ; y[i] = abs(x[i]-x[i+1])

        Addu    r5 , r5 , 4
        Addu    r4 , r4 , 4
        Bne     r4 , r6 , Loop
```

Version 2 :

```
Loop:  Lw      r8 , 0 (r4 )           ; x[i ]
        Lw      r9 , 4 (r4 )         ; x[i+1]
        Sub     r10, r8, r9

        Slt     r11, r10, r0
        Sub     r12, r0 , r11
        Xor     r10, r10, r12
        Add     r10, r10, r11

        Sw      r10, 0 (r5 )         ; y[i] = abs(x[i]-x[i+1])

        Addu    r5 , r5 , 4
        Addu    r4 , r4 , 4
        Bne     r4 , r6 , Loop
```

Dans tout l'exercice, on suppose que dans le tableau x , la probabilité que $x[i] < x[i+1]$ est de 40%.

- 1- Le découpage du pipeline en 9 étages a-t-il un impact sur le compilateur ?
- 2- Dans le processeur Mips, il existe des dépendances de données d'ordre 1, 2 et 3. Dans P9 quelles sont les dépendances de données ?
- 3- Quels sont les bypass nécessaires à l'exécution des instructions dans ce P9 ? Illustrer pour chaque bypass son utilisation à l'aide d'un exemple d'une suite d'instructions
- 4- Pour la Version 1, modifier le code de la boucle pour qu'il soit exécutable sur la réalisation **P9**. Analyser l'exécution de cette boucle à l'aide d'un schéma simplifié dans l'hypothèse où le branchement échoue. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 5- Pour la Version 2, modifier le code de la boucle pour qu'il soit exécutable sur la réalisation **P9**. Analyser l'exécution de cette boucle à l'aide d'un schéma simplifié. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 6- Pour chacune des deux versions, proposer un code optimisé en modifiant l'ordre des instructions. Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles de gel sur le code après optimisation. Pour chacune des deux versions, calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 7- Pour chacune des deux versions, dérouler une fois le code de la boucle (traitement de 2 éléments à chaque itération) et optimiser. Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles de gel sur le code après optimisation. Pour chacune des deux versions, calculer le nombre moyen de cycles par itération, le CPI et le CPI-utile.
- 8- Pour chacune des deux versions, optimiser le code de la boucle à l'aide de la technique « software pipeline ». Décrivez clairement votre démarche. Précisez le nombre d'étages et les instructions qui composent chaque étage. Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles de gel sur le code après optimisation. Pour chacune des deux versions, calculer le nombre moyen de cycles par itération, le CPI et le CPI-utile.